

flashjet

GPU-native jet clustering with the full merge history

Chirayu Gupta

ML4Jets 2026 · Vienna

September 2026

Benchmarks: **ATLAS Open Data** (CC0). Closures: analytic predictions on toy showers.

- 1 Jet clustering, and flashjet
- 2 Correctness
- 3 Speed
- 4 Conclusions

Jet clustering, and flashjet

Jet clustering: the workhorse, and its cost

Sequential recombination — anti- k_t , k_t , Cambridge–Aachen, as implemented in **FastJet** — is the LHC’s standard jet definition. It is **infrared and collinear safe**, which is why it survived, and it is $O(N \log N)$ **but strictly sequential**: each merge depends on the one before it.

That sequential structure is exactly what has kept it on the **CPU** while the rest of the pipeline moved to accelerators.

And it is rarely run once. The same event is clustered again for every jet definition, every radius, every systematic variation — and, in a training loop, once per epoch.

How often clustering is re-run

context	per event
reconstruction	a few
small- R , large- R , trimmed	
JES/JER systematics	$\mathcal{O}(10\text{--}100)$
substructure (2nd pass)	$\times 2$
R - / grooming scan	$\mathcal{O}(10)$
ML training	1 / epoch

Each row multiplies the one above it. The cost is not one clustering — it is the same event clustered many times over.

What it is. A GPU-native implementation of the generalised- k_t family — anti- k_t ($p=-1$), k_t ($p=+1$), Cambridge–Aachen ($p=0$) — written in **Triton/PyTorch**.

How it works. A batch of jets or events is clustered in **device memory**, and the data never returns to the host during clustering. The merge history is recorded **in-kernel**, so the full clustering tree comes back with the jets.

Backends, dispatched automatically by problem size:

- a fused register kernel for small N
- a scratch nearest-neighbour Triton kernel for large N
- a pure-torch fallback — so the same call runs on CPU

Design choices

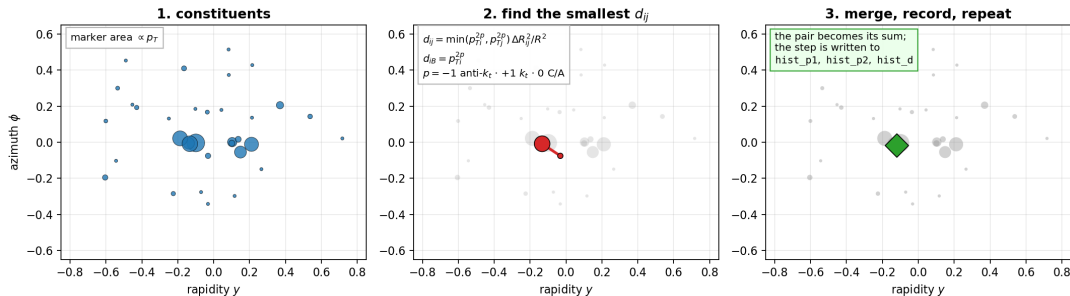
- **batched**: many jets at once, ragged input via a mask
- **history kept**: substructure becomes an array read, not a second clustering pass
- **one entry point**: identical call on CPU and GPU

Status

A **prototype** — validated, but a Python/Triton library, not integrated into experiment software.

How sequential recombination works

Sequential recombination — and what flashjet keeps from it

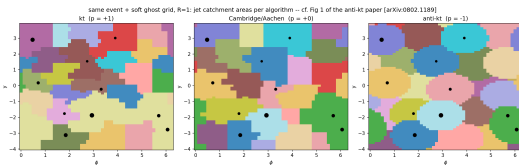


Repeat until nothing is left: if the smallest distance is a d_{ij} , merge that pair; if it is a d_{iB} , call that a jet and remove it.

One exponent p selects the algorithm — the *same* kernel serves all three.

One exponent, three algorithms

p	algorithm	merges first	gives
-1	anti- k_t	around the hardest particles	rigid, circular jets
0	Cambridge–Aachen	the closest pair	angular-ordered tree
+1	k_t	the softest pair	k_t -ordered tree



The ordering is not a detail — it is what the substructure reads.

- anti- k_t gives the *jets*: catchment areas $\rightarrow \pi R^2$ — the defining rigid-cone result, from flashjet's own clustering.
- **C/A** orders by angle $\Rightarrow$ the natural tree for **grooming** and the **Lund plane**.
- k_t orders by relative transverse momentum $\Rightarrow$ the natural tree for **exclusive subjets**.

So flashjet keeps whichever history the observable needs — and all three cost the same.

It returns the tree, not just the jets

Clustering a batch returns **two** things — the jets, and the record of how they were built.

field	shape	what it holds
jet_idx	(B, N)	which jet each particle ended up in
n_jets	$(B,)$	how many jets were found
hist_p1, hist_p2	(B, N)	which pair merged , at each step
hist_child	(B, N)	what they merged <i>into</i>
hist_d	(B, N)	the distance d_{ij} at that step

Why the second half matters

The bottom three rows are the **complete binary merge tree**. Groomed mass, z_g , R_g , Lund coordinates and exclusive subjets are all **coordinates on that tree** — so they become array reads, with **no second clustering pass**.

Using it

```
import flashjet

out = flashjet.cluster(p4, mask, R=1.0, algorithm="antikt")
```

argument	type	
p4	$(B, N, 4)$ tensor	(p_x, p_y, p_z, E) ; CPU or CUDA
mask	(B, N) bool	marks real constituents in a padded batch
R	float	jet radius
algorithm	str	"antikt" "kt" "cambridge"
p	float	generalised- k_t exponent (overrides algorithm)

out.n_jets	# (B,)	jets found
out.jets_p4(p4)	# (B,J,4)	jet four-momenta
out.splitting_scales()	#	$\sqrt{d_{12}}$, $\sqrt{d_{23}}$, ...
out.exclusive_jets(n_jets=3)	# F1	exclusive k_t subjects
out.groomed_jets(p4, R, z_cut=0.1, beta=0.0)	# F2	soft drop / mMDT
out.lund_coordinates(p4, R)	# F3	primary Lund plane

The same call runs unchanged on CPU and GPU — the backend is chosen from the problem size, not by the caller.

What is implemented

Clustering

- generalised- k_t : anti- k_t , k_t , C/A
- batched, ragged input via a mask
- three backends, auto-dispatched by size
- merge history recorded in-kernel
- jet regime *and* event regime

Substructure — reads of the history

- **F1** exclusive k_t subjets
- **F2** soft drop / mMDT / mass-drop
- **F3** primary Lund coordinates

All three are pure-torch post-reads: no kernel changes, CPU/GPU identical, negligible cost.

It is differentiable

`jets_p4` is a `scatter_add` of the constituents onto their jets, so **gradients flow through the clustering** — the jet four-momenta can sit inside a loss without a custom backward pass.

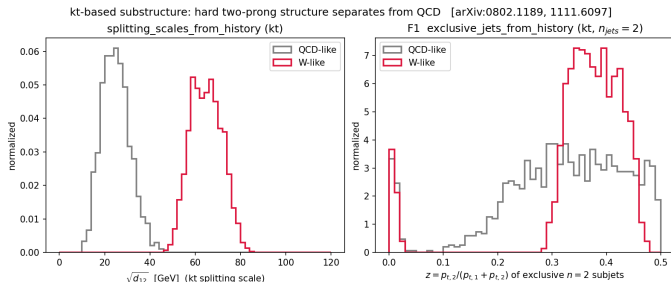
F1 — exclusive subjects from the k_t history

What it does. Undo the last merges of the sequence to expose exactly n subjects — or stop at a d_{cut} .

How. The history is already a binary tree. Walk back up it $n - 1$ steps; no reclustering.

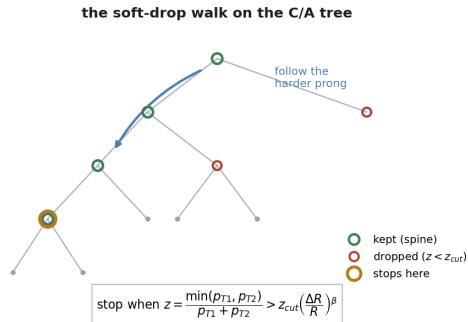
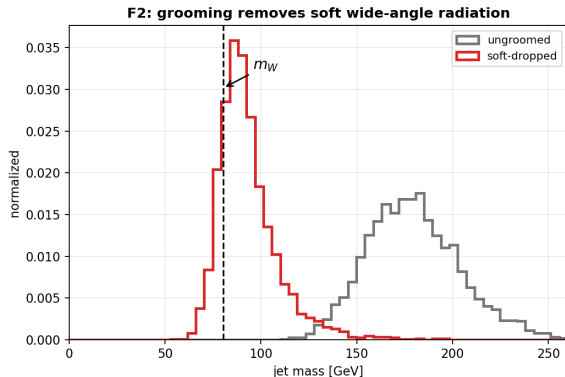
Why k_t . k_t merges the softest pair first, so the *last* merges are the hard prongs — exactly what a 2-prong decay leaves behind.

The splitting scales $\sqrt{d_{12}}, \sqrt{d_{23}}, \dots$ fall out of the same array.



Toy W-like (2-prong) vs QCD-like fat jets. **Left:** the k_t splitting scale $\sqrt{d_{12}}$ separates cleanly. **Right:** the momentum balance z of the two exclusive subjects — the W sits near $z \sim 0.4$, QCD spreads to small z .

F2 — soft drop / mMDT grooming from the C/A history



Walk down the C/A tree along the harder branch, discarding the softer prong until the momentum-sharing condition is met. $\beta = 0$ recovers mMDT.

The walk is $O(\log_2 n)$ on the stored tree — it never touches the constituents again, so grooming is essentially free once clustered.

What the tree looks like on real jets

C/A merge trees of real jets — root at top, constituents at the bottom

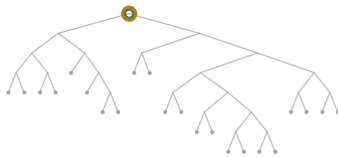
ATLAS Top Tagging Open Data · vertical = declustering depth · horizontal = angular ordering

Light quark / gluon jet



a long spine: soft prong after soft prong
is stripped, and the mass collapses
 $n_{\text{drop}} = 8$ $m: 80 \rightarrow 1 \text{ GeV}$

Boosted top — a clean two-prong core



the very first split is already balanced,
so nothing is groomed away
 $n_{\text{drop}} = 0$ $m: 80 \rightarrow 80 \text{ GeV}$

Boosted top — the full $t \rightarrow bW$ system



also stops at once, but on a wider split,
so the whole decay is kept
 $n_{\text{drop}} = 0$ $m: 170 \rightarrow 170 \text{ GeV}$

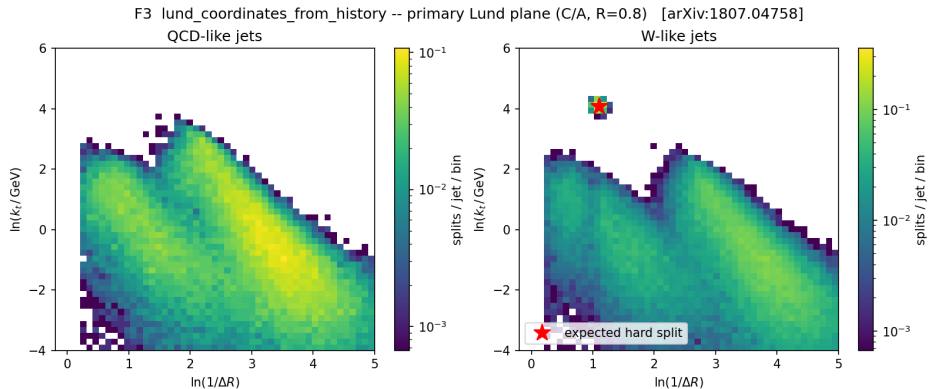
● soft-drop spine ● dropped prong ● grooming stops here ● constituent

A jet without a decay has nothing to stop on. The light-quark/gluon jet is a long *staircase*: soft prong after soft prong is stripped, and its mass collapses to nothing.

ATLAS Open Data, C/A with soft drop ($z_{\text{cut}}=0.1$, $\beta=0$). **The number of drops is itself a discriminant** — and it is read straight off the stored tree, at no extra cost.

A real decay stops at once. Both top jets halt on their very first split, because it is already hard and balanced — so the mass survives.

F3 — primary Lund coordinates from the C/A history



What. Every primary splitting emits $(z, \Delta R, k_t, \ln 1/\Delta R, \ln k_t, d)$.

How. One pass down the primary branch — each node is one point in the plane.

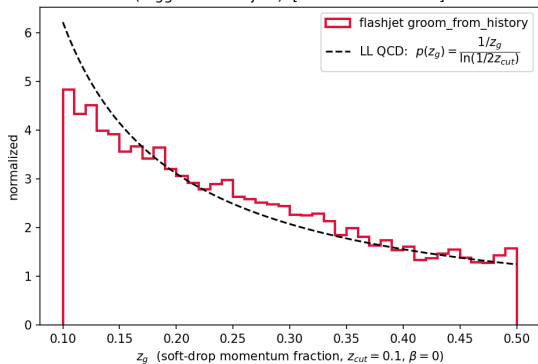
Check. The d -channel ties *exactly* to the splitting scales computed independently.

QCD fills the soft-collinear region smoothly; the W-like sample adds a **localised hard splitting** exactly where the decay is predicted to sit (star).

Correctness

The history is right: closures against analytic predictions

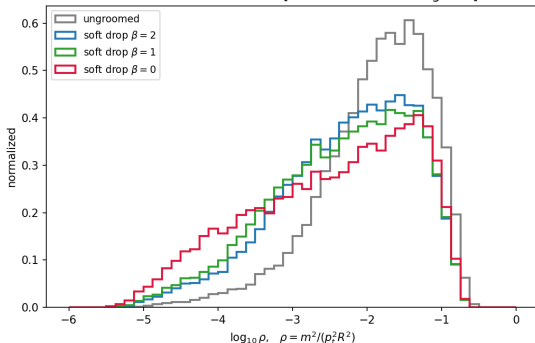
groomed splitting fraction vs the analytic prediction
(tagged 72% of jets) [cf. arXiv:1402.2657]



Soft-drop z_g follows the $1/z$ form predicted at leading-log, recovered from the C/A history.

Toy-shower input, so the *prediction* is unambiguous — these test the decoders, not an event generator.

groomed jet mass: stronger grooming (smaller β) pushes the soft-radiation mass down [cf. arXiv:1402.2657 Figs 3-4]

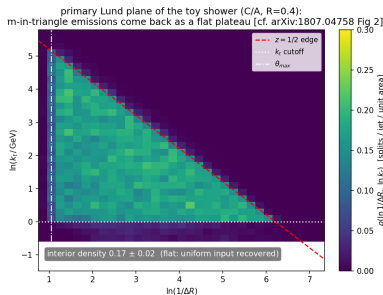


Groomed-mass ordering in β : larger β grooms less, so the mass peak moves up.

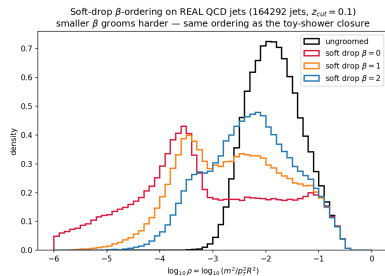
Clustering geometry



anti- k_t jet areas $\rightarrow \pi R^2$.



Lund plane uniform inside the kinematic triangle.



β -family ordering.

Independent NumPy tree-walks pin every decoder; the d -channel of the Lund output ties exactly to the splitting scales.

Same jets as FastJet — across the whole grid

3 algorithms $\times$ **5 radii** $\times$ **5 multiplicity bins** $\times$ **2 samples**, 20 000 jets per bin, on ATLAS Open Data.

check	result
number of jets found	100.000 % at every one of 150 points
leading-jet p_T within 10^{-4}	1.0000 at 148/150 points
median relative difference	$\sim 7 \times 10^{-8}$

k_t and C/A had never been checked against FastJet on real detector data — only against NumPy tree-walks. They now agree across the full radius $\times$ multiplicity grid.

The two exceptions are the *same single jet*: one constituent assigned across an R boundary. Jet counts still match and the leading jet is never ambiguous.

Same jets as the experiment

A stronger test than agreeing with FastJet.

ATLAS publishes an open dataset carrying both the **calorimeter clusters** and **its own reconstructed jets**. Cluster the ~ 600 clusters per event ourselves, and compare:

n_{jets} vs ATLAS	100.0 % match
mean jets/event	10.96 vs 10.96

This closes the algorithm against **the experiment's own published output** — not against another implementation of the same algorithm.

599.3 clusters/event: the event regime, where the problem is $O(N^2)$.

Speed

The benchmark

Data — ATLAS Open Data, freely presentable

sample	jets	$\langle n_{\text{const}} \rangle$
boosted top	30 000	59.2
QCD (light q/g)	30 000	52.4

Large- R jets with calorimeter-cluster constituents. The grid is **3 algorithms** $\times$ **5 radii** $\times$ **5 multiplicity bins**, 20 000 jets per bin.

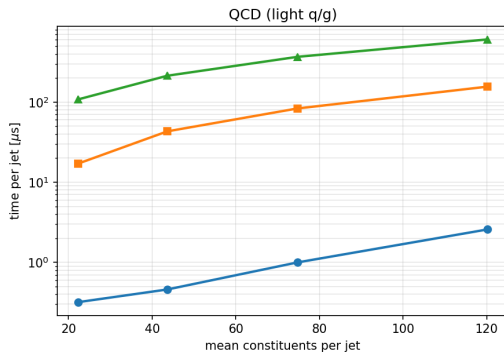
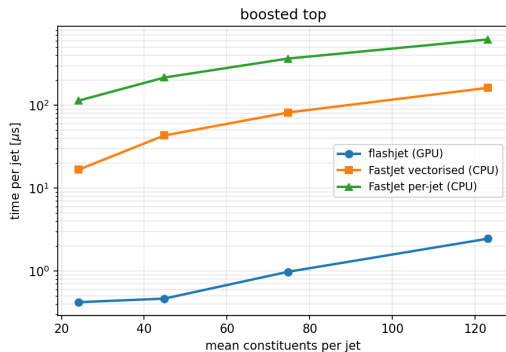
Method

- 1 extract constituents once, save;
- 2 cluster on GPU, time it, save the *same* events;
- 3 cluster those saved events with FastJet;
- 4 **check they agree** — else the timing is meaningless.

Both clusterers read the **same saved file**. Speedups are quoted against the **vectorised** FastJet interface — the faster, fairer baseline.

Time per jet

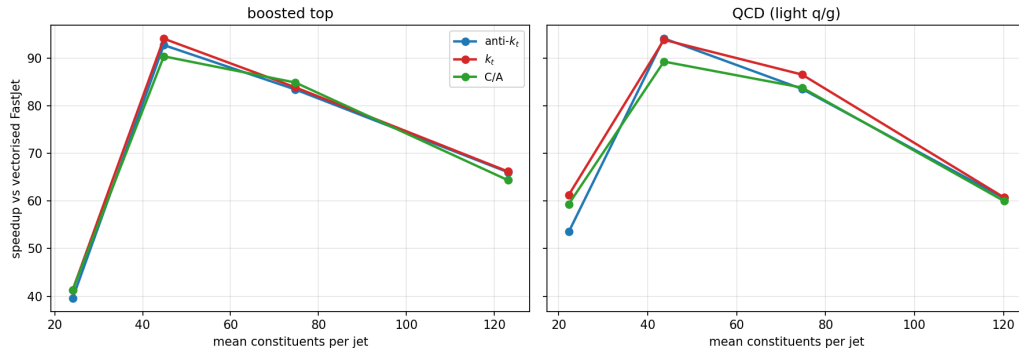
ATLAS Top Tagging Open Data — anti- k_t $R = 1.0$



Log scale — each gridline is 10×. flashjet stays nearly flat while both FastJet interfaces climb steeply with multiplicity: **0.35–2.7 μs per jet** against **17–155 μs** for vectorised FastJet, and **110–620 μs** per jet one at a time. Benchmarked on an NVIDIA V100-class GPU.

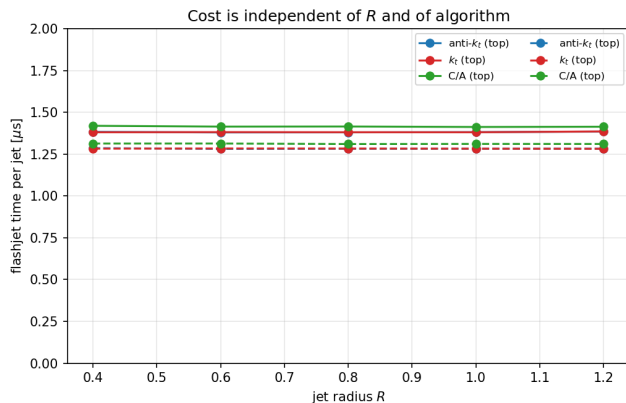
The advantage grows with jet complexity

Speedup vs multiplicity — $R = 1.0$ (all points 100% n_{jets} agreement)



All three algorithms, both samples, $R = 1.0$. **39–99 $\times$** over vectorised FastJet across the grid (median **64 $\times$**), and every point has **100 %** n_{jets} agreement. Busier jets $\rightarrow$ bigger win — the regime that matters for boosted-object physics.

Cost is independent of R — and of the algorithm



Across $R = 0.4 \rightarrow 1.2$: a **0.3 % spread** over a $3\times$ range in R .

R changes *which* pairs merge, not *how many* distances are computed.

Across algorithms: anti- k_t , k_t and C/A agree to within $\sim 4\%$ at every multiplicity.

k_t and C/A timing is **free**.

Conclusions

- 1 **Clusters on the GPU and keeps the full merge history**, so substructure — exclusive subjects, soft drop, Lund coordinates — comes free as array reads.
- 2 **Gives the same jets as FastJet**. 150/150 grid points at 100 % n_{jets} agreement, across anti- k_t/k_t /C-A, $R = 0.4\text{--}1.2$ and 24–123 constituents per jet. k_t and C/A validated on real data for the first time.
- 3 **Reproduces the experiment's own reconstructed jets** exactly, at ~ 600 clusters per event.
- 4 **39–99 $\times$ faster** than vectorised FastJet, and the advantage *grows* with jet complexity. Cost is independent of R and of algorithm.
- 5 Still a **prototype**, with meaningful headroom left.

Thank you

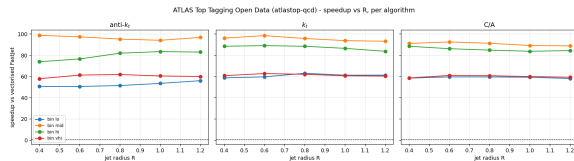
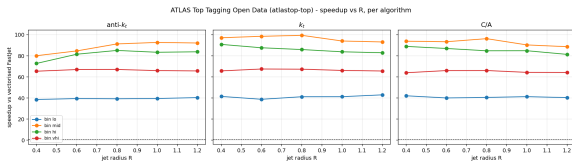
Questions?

Benchmarks: ATLAS Open Data (CC0).

Physics closures: analytic predictions on toy showers.

Backup

Backup — speedup vs R , per algorithm



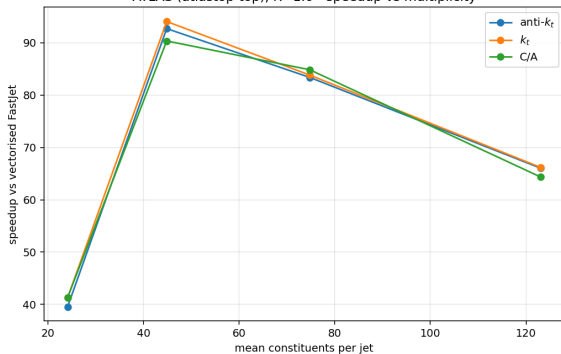
boosted top

Flat in R for all three algorithms; ordering is by multiplicity bin.

QCD (light q/g)

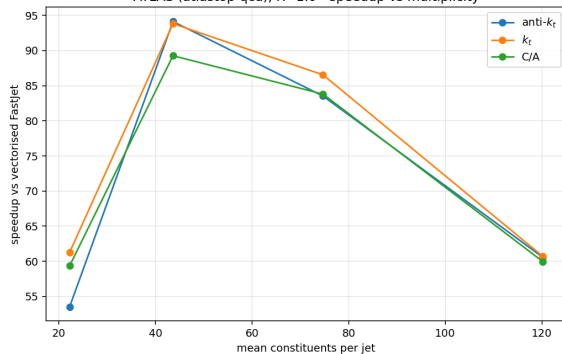
Backup — speedup vs multiplicity

ATLAS (atlastop-top), R=1.0 - speedup vs multiplicity



boosted top

ATLAS (atlastop-qcd), R=1.0 - speedup vs multiplicity



QCD (light q/g)

Backup — the numbers

sample	alg	$\langle n \rangle$	flashjet [μ s/jet]	FastJet vec. [μ s/jet]	speedup
top	anti- k_t	24.1	0.427	16.5	39×
top	anti- k_t	44.8	0.464	43.0	93 ×
top	C/A	74.7	1.000	84.8	85×
top	C/A	123.1	2.565	165.1	64×
qcd	anti- k_t	22.2	0.346	17.5	51×
qcd	anti- k_t	43.6	0.463	45.7	99 ×
qcd	anti- k_t	74.7	1.215	89.8	74×
qcd	anti- k_t	120.2	2.673	154.9	58×

39–99× over vectorised FastJet (median 64×), with 100 % jet-count agreement everywhere. Benchmarked on an **NVIDIA V100-class** GPU.

$R = 1.0$ rows shown. Against the per-jet FastJet interface the gap is roughly *two orders of magnitude*.

All benchmark data is **ATLAS Open Data** (CC0), read over the public redirector — no grid certificate required.

dataset	used for
ATLAS Top Tagging Open Data	timing + FastJet agreement
2020 ATLAS Jet Reconstruction	closure vs the experiment's own jets

Constituents are calorimeter clusters, massless. Physics closures use toy showers compared against leading-log analytic predictions, so the prediction being tested is unambiguous.